



Penjelasan UML Diagrams

ESP32 Peltier Cooling
Early Warning System (EWS)

Project: Solar Panel Fire Early Warning System

Stack: Node.js · Express · Socket.IO · Firebase · Linear Regression

Hardware: ESP32 + DHT22 Sensor + Peltier Cooler

Tanggal: Mei 2026

Daftar Isi

1. Pendahuluan — Gambaran Umum Sistem

2. Diagram 1: System Architecture Diagram — Arsitektur dan Komponen Sistem

3. Diagram 2: Sequence Diagram — Alur Pemrosesan Data (processEarlyWarning)

4. Diagram 3: Class & Module Diagram — Struktur Modul dan Fungsi

5. Diagram 4: Flowchart Danger Classification — Logika Klasifikasi Bahaya

6. Kesesuaian Implementasi dengan Rancangan UML

1. Pendahuluan — Gambaran Umum Sistem

Early Warning System (EWS) untuk Panel Surya adalah sistem peringatan dini berbasis *Machine Learning* (Linear Regression) yang dirancang untuk mendeteksi potensi kebakaran pada panel surya akibat *overheating*. Sistem ini menggunakan sensor suhu dan kelembaban DHT22 yang terpasang pada modul ESP32, dan memanfaatkan pendinginan Peltier sebagai mekanisme mitigasi otomatis.

Tujuan Utama Sistem

- 1. Mengumpulkan data suhu & kelembaban secara real-time dari sensor DHT22 via ESP32
- 2. Memprediksi tren suhu ke depan menggunakan algoritma **Linear Regression**
- 3. Mengklasifikasikan level bahaya (Normal → Warning → Danger → Critical)
- 4. Mengirimkan notifikasi peringatan melalui **Telegram Bot** dan **Web Dashboard**
- 5. Mengontrol sistem pendinginan Peltier secara otomatis maupun manual

Komponen Utama Sistem

Hardware

ESP32 microcontroller dengan sensor DHT22 (suhu + kelembaban) dan modul Peltier Cooler. ESP32 membaca sensor dan mengirim data ke Firebase.

`prototype_server.ino`

Database

Firebase Realtime Database sebagai jembatan data antara ESP32 (hardware) dan Node.js server (backend). Menyimpan data sensor, riwayat, alert, dan kontrol.

Backend

Node.js server dengan Express + Socket.IO. Menjalankan analisis Linear Regression, klasifikasi bahaya, dan mengelola notifikasi secara periodik.

`src/server.js`

Frontend

Web Dashboard real-time yang menampilkan data sensor, grafik tren suhu, prediksi, level bahaya, dan kontrol Peltier. Update via Socket.IO.

`public/index.html`

Technology Stack

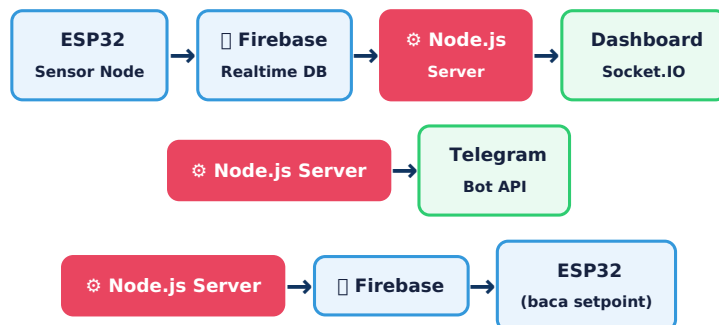
Layer	Teknologi	Fungsi
Hardware	ESP32 + DHT22 + Peltier	Akuisisi data sensor & kontrol pendinginan
Database	Firebase Realtime Database	Penyimpanan data real-time & sinkronisasi
Backend	Node.js, Express, Socket.IO, node-cron	Pemrosesan data, ML, API, notifikasi
ML Engine	Linear Regression (custom)	Prediksi tren suhu & estimasi waktu kritis

Notifikasi	Telegram Bot API	Alert darurat ke pengguna via chat
Dashboard	HTML/CSS/JS + Chart.js + Socket.IO Client	Visualisasi data real-time

2. Diagram 1: System Architecture Diagram

Diagram arsitektur sistem ini menunjukkan **komponen-komponen utama** dalam sistem dan bagaimana mereka saling **berkomunikasi**. Diagram ini berbentuk *graph* yang menggambarkan aliran data dari hardware (ESP32) hingga ke antarmuka pengguna (Dashboard & Telegram).

2.1 Alur Komunikasi Antar Komponen



2.2 Penjelasan Setiap Koneksi

Dari	Ke	Jenis	Penjelasan
ESP32	Firebase	Push Data	ESP32 membaca sensor DHT22 secara periodik, lalu mengirim data suhu & kelembaban ke path <code>/devices/{device_id}/latest</code> dan <code>/history</code> di Firebase.
Firebase	Node.js Server	Read Data	Server membaca data historis (<code>getHistoricalData</code>) dan data terkini (<code>getCurrentReading</code>) dari Firebase untuk dianalisis.
Node.js Server	Firebase	Write Control	Server menulis perintah kontrol (target suhu, mode auto) ke path <code>/control/{device_id}</code> agar ESP32 dapat membacanya.
Firebase	ESP32	Read Setpoint	ESP32 membaca target suhu dan mode auto dari <code>/control/{device_id}</code> untuk menentukan apakah Peltier harus ON/OFF.
Node.js Server	Web Dashboard	Socket.IO	Server menyiarkan (<code>emit</code>) data terbaru ke semua dashboard yang terhubung via event <code>'sensor-update'</code> .
Web Dashboard	Node.js Server	HTTP API	Dashboard mengirim request ke REST API server (contoh: <code>POST /api/control</code>) untuk mengubah pengaturan.
Node.js Server	Telegram	HTTPS API	Saat terdeteksi bahaya, server mengirim notifikasi ke Telegram Bot API dengan format MarkdownV2.

2.3 Struktur Internal Node.js Backend

Backend terdiri dari 5 file/modul utama yang saling terhubung. `server.js` sebagai orchestrator yang mengimpor dan mengoordinasikan seluruh modul:

Modul	File	Peran
Server (Main)	<code>server.js</code>	Orchestrator utama. Menjalankan Express server, Socket.IO, cron scheduler, dan memanggil semua modul lain dalam fungsi <code>processEarlyWarning()</code> .
Linear Regression	<code>linearRegression.js</code>	Engine Machine Learning. Melatih model regresi linear, memprediksi suhu, dan menghitung waktu menuju threshold bahaya.

Alert System	<code>alertSystem.js</code>	Klasifikasi bahaya berdasarkan suhu, kelembaban, dan prediksi. Menghasilkan objek alert dengan pesan terformat.
Firestore Service	<code>firebaseService.js</code>	Abstraksi akses ke Firestore Realtime Database. Menangani CRUD data sensor, kontrol, prediksi, dan alert.
Telegram Service	<code>telegramService.js</code>	Pengiriman notifikasi ke Telegram dengan fitur rate limiting per-level agar tidak spam.

Hubungan Dependency

`server.js` → mengimpor dan menggunakan → `linearRegression.js` , `alertSystem.js` , `firebaseService.js` , `telegramService.js` . Tidak ada dependency antar modul pendukung (loose coupling).

3. Diagram 2: Sequence Diagram — Alur Pemrosesan Data

Diagram sekuens ini memvisualisasikan **alur eksekusi fungsi utama** `processEarlyWarning()` yang dijalankan secara periodik oleh *cron scheduler*. Fungsi ini merupakan jantung dari sistem EWS — mengkoordinasikan seluruh pipeline dari pengambilan data hingga penyiaran notifikasi.

3.1 Participant (Aktor dalam Sequence)

Participant	Representasi	Peran
Scheduler	<code>node-cron</code>	Pemicu otomatis. Menjalankan analisis setiap 1 menit (produksi) atau 5 detik (demo).
Server	<code>server.js</code>	Orchestrator. Menerima trigger dan mengkoordinasikan seluruh proses.
Firebase	<code>firebaseService.js</code>	Penyedia data. Mengambil dan menyimpan data dari/ke Firebase.
LR	<code>LinearRegression</code>	Engine ML. Melatih model dan menghasilkan prediksi.
Alert	<code>alertSystem.js</code>	Pengklasifikasi. Menentukan level bahaya dan format notifikasi.
Telegram	<code>telegramService.js</code>	Notifier. Mengirim pesan ke Telegram jika ada bahaya.
Dashboard	Web Dashboard	UI. Menerima update real-time via Socket.IO.

3.2 Langkah-langkah Eksekusi (Step-by-Step)

Step	Dari → Ke	Aksi	Penjelasan
1	Cron → Server	<code>processEarlyWarning()</code>	Scheduler memicu fungsi utama. Di mode produksi, ini berjalan setiap 1 menit via <code>node-cron</code> . Di mode demo, setiap 5 detik via <code>setInterval</code> .
2	Server → Firebase	<code>getHistoricalData(deviceId, 30)</code>	Mengambil 30 data point terakhir dari <code>/devices/{id}/history</code> . Data diurutkan berdasarkan timestamp dan dinormalisasi dari <code>millis()</code> ESP32 ke Unix timestamp.
3	Firebase → Server	Return <code>history[]</code>	Array berisi 30 objek data sensor (suhu, kelembaban, timestamp) yang sudah dinormalisasi.
4	Server → Firebase	<code>getCurrentReading(deviceId)</code>	Mengambil data sensor terkini dari <code>/devices/{id}/latest</code> .
5	Firebase → Server	Return <code>currentReading</code>	Objek data sensor terakhir (suhu aktual, kelembaban, status Peltier, dll).
6	Server → LR	<code>train(history)</code>	Melatih model Linear Regression dengan data historis. Menghitung slope (laju kenaikan suhu per menit), intercept , dan R² (akurasi model).
7	LR → Server	Return <code>modelStats</code>	Objek berisi: slope, intercept, rSquared, interpretation, dan calculationLogs.
8	Server → LR	<code>predict(PREDICTION_MINUTES)</code>	Memprediksi suhu N menit ke depan (default: 10 menit) berdasarkan model yang sudah dilatih. Menggunakan formula: $y = mx + b$.

9	LR → Server	Return <code>prediction</code>	Objek berisi: predictedTemperature, confidence ($R^2 \times 100$), trend (naik/turun/stabil), ratePerMinute.
10	Server → LR	<code>minutesToThreshold(50, currentTemp)</code>	Menghitung estimasi berapa menit lagi suhu mencapai threshold kritis (50°C). Formula: $t = (\text{threshold} - \text{current}) / \text{slope}$.
11	LR → Server	Return <code>minutesToDanger</code>	Angka menit (atau <code>null</code> jika suhu sedang turun / stabil).
12	Server → Alert	<code>classifyDanger(current, prediction, modelStats)</code>	Mengklasifikasikan level bahaya berdasarkan 3 faktor: suhu aktual, prediksi suhu, dan kelembaban. Menghasilkan level tertinggi dari ketiga evaluasi.
13	Alert → Server	Return <code>dangerInfo</code>	Objek berisi: level (normal/warning/danger/critical), tempLevel, humidityRisk, predictionRisk, dan array factors.
14	Server → Alert	<code>generateAlert(dangerInfo, ...)</code>	Membuat objek alert terformat dengan judul, pesan, detail, dan tindakan yang diperlukan. Return <code>null</code> jika level = normal.
15	Alert → Server	Return <code>alertObject</code>	Objek alert lengkap (atau <code>null</code>). Berisi: level, title, message, action, sensorData, prediction, timestamp.
↳ Kondisional: Jika <code>alertObject !== null</code> (terdeteksi bahaya):			
16a	Server → Firebase	<code>saveAlert(deviceId, alertObject)</code>	Menyimpan alert ke Firebase di path <code>/devices/{id}/alerts</code> untuk logging.
16b	Server → Telegram	<code>notify(alertObject, ...)</code>	Mengirim notifikasi ke Telegram. Dilengkapi rate limiting agar tidak spam (warning: 30 menit, danger: 15 menit, critical: 5 menit).
↳ Jika <code>alertObject === null</code> (status normal): Memanggil <code>telegram.onStatusNormal()</code> untuk mereset recovery cooldown.			
17	Server → Dashboard	<code>io.emit('sensor-update', latestState)</code>	Menyiarkan seluruh state terkini (sensor, model, prediksi, bahaya, chart) ke semua dashboard yang terhubung via Socket.IO.

Catatan Penting

Seluruh proses dari Step 1 hingga Step 17 berjalan dalam satu siklus eksekusi `processEarlyWarning()`. Di mode produksi, siklus ini berulang setiap **1 menit**. Selain trigger periodik, siklus juga dipicu secara real-time oleh Firebase listener (`subscribeToSensor`) setiap kali ESP32 mengirim data baru.

4. Diagram 3: Class & Module Diagram

Meskipun sistem ini ditulis dalam JavaScript/Node.js (bukan bahasa OOP murni), diagram ini memetakan **struktur modul** sebagai Class Diagram. Hal ini berguna untuk melihat *property* dan *method* apa saja yang diekspos oleh setiap komponen.

4.1 Class: LinearRegression

File: linearRegression.js

Satu-satunya komponen yang menggunakan paradigma **Object-Oriented** (class ES6).
Diinstansiasi sebagai objek `model` di `server.js`.

Properties (Atribut)

Akses	Nama	Tipe	Penjelasan
Public	<code>slope</code>	Number	Koefisien kemiringan garis regresi (m). Menunjukkan laju kenaikan/penurunan suhu per menit. Nilai positif = suhu naik.
Public	<code>intercept</code>	Number	Titik potong sumbu Y (b). Representasi suhu awal saat $t=0$.
Public	<code>rSquared</code>	Number	Koefisien determinasi (R^2). Nilai 0-1 yang menunjukkan seberapa akurat model mewakili data. Semakin mendekati 1, semakin akurat.
Public	<code>dataPoints</code>	Array	Array data historis yang digunakan untuk training terakhir.
Public	<code>isTrained</code>	Boolean	Flag yang menunjukkan apakah model sudah dilatih dan siap untuk prediksi.
Public	<code>calculationLogs</code>	Object	Log detail perhitungan (sumX, sumY, sumXY, sumX2, dll) untuk transparansi dan debugging.

Methods (Fungsi)

Akses	Method	Return	Penjelasan
Public	<code>train(dataPoints)</code>	Object	Melatih model dengan data historis sensor. Menghitung slope & intercept menggunakan formula Least Squares: $m = (n\sum xy - \sum x \sum y) / (n\sum x^2 - (\sum x)^2)$. Minimal memerlukan 3 data point.
Public	<code>predict(minutesAhead)</code>	Object	Memprediksi suhu N menit ke depan menggunakan formula: $y = mx + b$. Mengembalikan predictedTemperature, confidence, trend, dan ratePerMinute.
Public	<code>minutesToThreshold(thresholdTemp, currentTemp)</code>	Number null	Menghitung estimasi waktu (menit) hingga suhu mencapai threshold tertentu. Formula: $t = (threshold - current) / slope$. Return null jika slope ≤ 0 (suhu tidak naik).
Public	<code>getRegressionLine(numPoints)</code>	Array	Menghasilkan titik-titik untuk menggambar garis regresi pada chart, termasuk 5 titik prediksi ke depan.

Private	<code>_calculateRSquared(points)</code>	Object	Menghitung R^2 (Coefficient of Determination). $R^2 = 1 - (SS_{res} / SS_{tot})$. Digunakan internal oleh <code>train()</code> .
Private	<code>_interpretSlope()</code>	String	Menginterpretasikan slope menjadi teks deskriptif (contoh: "Suhu naik cepat (~12.0°C/jam)").

4.2 Module: firebaseService

File: firebaseService.js

Modul fungsional yang mengabstraksi semua operasi Firebase Realtime Database. Diekspor sebagai kumpulan fungsi (bukan class).

Fungsi	Parameter	Return	Penjelasan
<code>initFirebase()</code>	—	void	Inisialisasi Firebase Admin SDK menggunakan service account dari environment variables.
<code>getCurrentReading(deviceId)</code>	deviceId: string	Object	Membaca data sensor terkini dari <code>/devices/{id}/latest</code> .
<code>getHistoricalData(deviceId, limitPoints)</code>	deviceId: string, limitPoints: number	Array	Mengambil N data historis terakhir dari <code>/devices/{id}/history</code> , dengan normalisasi timestamp.
<code>subscribeToSensor(deviceId, callback)</code>	deviceId: string, callback: Function	Function (unsubscribe)	Mendengarkan perubahan real-time pada data sensor. Return fungsi unsubscribe.
<code>getControlState(deviceId)</code>	deviceId: string	Object	Membaca status kontrol (target_temperature, auto_mode) dari <code>/control/{id}</code> .
<code>setControl(deviceId, controlData)</code>	deviceId: string, controlData: Object	Object	Menulis setpoint kontrol baru ke Firebase agar ESP32 dapat membacanya.
<code>savePrediction(deviceId, predictionData)</code>	deviceId: string, predictionData: Object	void	Menyimpan hasil prediksi ke <code>/devices/{id}/predictions</code> (logging).
<code>saveAlert(deviceId, alertData)</code>	deviceId: string, alertData: Object	void	Menyimpan alert ke <code>/devices/{id}/alerts</code> .
<code>generateMockData(baseTemp, trend)</code>	baseTemp: number, trend: string	Object	Menghasilkan data sensor simulasi untuk mode demo (rising/stable/cooling).

4.3 Module: alertSystem

File: alertSystem.js

Modul yang bertugas mengklasifikasikan level bahaya dan menghasilkan objek alert.

Ekspor	Tipe	Penjelasan
<code>THRESHOLDS</code>	Object	Konstanta threshold untuk suhu (normal <30°C, warning 30–40°C, danger 40–50°C, critical ≥50°C), kelembaban (tooLow <10%, low <20%), dan slope (rapid >0.5°C/min, fast >0.2°C/min).
<code>classifyDanger(sensorData, prediction, modelStats)</code>	Function	Mengevaluasi 3 faktor (suhu aktual, prediksi, kelembaban) dan mengembalikan level bahaya tertinggi beserta faktor-faktornya.
<code>generateAlert(dangerInfo, sensorData, prediction, minutesToDanger)</code>	Function	Menghasilkan objek alert terformat dengan title, message, detail, action, timestamp, dan sensorData. Return <code>null</code> jika level normal.
<code>buildFactorList(...)</code> (private)	Function	Membangun array string faktor-faktor bahaya untuk ditampilkan dalam notifikasi.

4.4 Module: telegramService

File: telegramService.js

Singleton class yang mengelola pengiriman notifikasi ke Telegram dengan fitur rate limiting per level.

Method	Penjelasan
<code>notify(alert, prediction, modelStats, minutesToDanger)</code>	Entry point utama. Mengecek rate limit, membangun pesan MarkdownV2, dan mengirim ke semua Chat ID yang dikonfigurasi. Rate limit: Warning=30 menit, Danger=15 menit, Critical=5 menit.
<code>onStatusNormal()</code>	Dipanggil saat status kembali normal. Mencatat waktu recovery untuk mencegah spam saat suhu berfluktuasi naik-turun.
<code>getStatus()</code>	Mengembalikan status rate limit Telegram saat ini (untuk API endpoint <code>/api/telegram-status</code>).

4.5 Relasi Antar Modul

Relasi	Tipe	Penjelasan
<code>server.js</code> → <code>LinearRegression</code>	Instantiates	Server membuat instance <code>new LinearRegression()</code> sebagai variabel global <code>model</code> .
<code>server.js</code> → <code>firebaseService</code>	Imports & Uses	Server mengimpor modul dan memanggil fungsi-fungsinya untuk akses database.
<code>server.js</code> → <code>alertSystem</code>	Imports & Uses	Server mengimpor <code>classifyDanger</code> , <code>generateAlert</code> , dan <code>THRESHOLDS</code> .
<code>server.js</code> → <code>telegramService</code>	Imports & Uses	Server mengimpor singleton dan memanggil <code>notify()</code> , <code>onStatusNormal()</code> , <code>getStatus()</code> .

5. Diagram 4: Flowchart — Logika Klasifikasi Bahaya

Flowchart ini menggambarkan secara spesifik bagaimana fungsi `classifyDanger()` di `alertSystem.js` menentukan **level bahaya akhir** berdasarkan penggabungan tiga faktor evaluasi: suhu saat ini, prediksi suhu, dan kelembaban.

5.1 Tahap 1: Evaluasi Suhu Saat Ini

Langkah pertama adalah mengevaluasi suhu aktual yang dibaca dari sensor DHT22:

Kondisi	Level	Keterangan
Suhu $\geq 50^{\circ}\text{C}$	CRITICAL	Suhu ekstrem. Risiko kebakaran sangat tinggi. Komponen bisa rusak permanen.
Suhu $\geq 40^{\circ}\text{C}$	DANGER	Overheat. Panel surya beroperasi di luar batas aman. Penurunan efisiensi drastis.
Suhu $\geq 30^{\circ}\text{C}$	WARNING	Mulai panas. Suhu melewati batas nyaman. Perlu pemantauan lebih ketat.
Suhu $< 30^{\circ}\text{C}$	NORMAL	Suhu aman. Panel beroperasi dalam batas normal (suhu ruang / pendinginan aktif).

5.2 Tahap 2: Evaluasi Prediksi Suhu

Langkah kedua menggunakan hasil prediksi dari model Linear Regression. Ini memberikan peringatan proaktif — sebelum suhu benar-benar mencapai level berbahaya:

Kondisi	Risk Level	Keterangan
Prediksi $\geq 50^{\circ}\text{C}$	CRITICAL	Model memprediksi suhu akan mencapai zona kritis dalam waktu dekat.
Prediksi $\geq 40^{\circ}\text{C}$	DANGER	Model memprediksi suhu akan memasuki zona bahaya.
Prediksi $\geq 30^{\circ}\text{C}$	WARNING	Model memprediksi suhu akan meningkat ke zona waspada.
Prediksi $< 30^{\circ}\text{C}$	NONE	Tidak ada risiko dari prediksi. Suhu diprediksi tetap aman.

⚠ Mengapa Evaluasi Prediksi Penting?

Tanpa prediksi, sistem hanya bisa bereaksi *setelah* suhu sudah berbahaya. Dengan Linear Regression, sistem bisa memberikan peringatan **sebelum** suhu mencapai batas kritis — memberikan waktu bagi operator untuk mengambil tindakan pencegahan.

5.3 Tahap 3: Evaluasi Kelembaban

Kelembaban rendah meningkatkan risiko kebakaran karena kondisi kering mempercepat penyebaran api:

Kondisi	Risk Level	Keterangan
Kelembaban $< 10\%$	CRITICAL	Sangat kering. Risiko kebakaran meningkat drastis. Bahan mudah terbakar.
Kelembaban $< 20\%$	WARNING	Kering. Kondisi perlu diwaspadai, terutama jika dikombinasikan dengan suhu tinggi.
Kelembaban $\geq 20\%$	NORMAL	Kelembaban cukup. Tidak ada risiko tambahan dari faktor kelembaban.

5.4 Tahap 4: Penentuan Level Akhir

Setelah ketiga evaluasi selesai, sistem menentukan **level bahaya akhir** dengan mengambil **level tertinggi** dari ketiga faktor. Urutan prioritas:



Contoh Skenario:

Suhu Aktual	Prediksi	Kelembaban	tempLevel	predictionRisk	humidityRisk	Level Akhir
25°C	28°C	60%	normal	none	normal	NORMAL
32°C	38°C	45%	warning	warning	normal	WARNING
28°C	45°C	55%	normal	danger	normal	DANGER
35°C	42°C	8%	warning	danger	critical	CRITICAL
52°C	58°C	15%	critical	critical	warning	CRITICAL

Catatan Penting

Perhatikan skenario ke-3: meskipun suhu aktual masih **28°C (normal)**, karena prediksi menunjukkan suhu akan mencapai **45°C (danger)**, sistem sudah memberikan peringatan **DANGER**. Ini adalah keunggulan utama pendekatan *Early Warning* berbasis Machine Learning.

5.5 Tindakan Berdasarkan Level

Level	Respons Sistem	Pesan ke Pengguna
NORMAL	Tidak ada notifikasi. Status ditampilkan di dashboard. Telegram recovery cooldown dimulai.	—
WARNING	Alert disimpan ke Firebase. Telegram dikirim (maks 1x/30 menit). Dashboard diperbarui.	"⚠ Peringatan: Suhu Meningkat — Periksa ventilasi dan pastikan panel tidak terhalang."
DANGER	Alert disimpan. Telegram dikirim (maks 1x/15 menit). Dashboard diperbarui dengan warna merah.	"BAHAYA: Suhu Tinggi Terdeteksi — SEGERA periksa panel surya!"
CRITICAL	Alert disimpan. Telegram dikirim (maks 1x/5 menit, frekuensi tertinggi). Dashboard menampilkan alarm.	"KRITIS: RISIKO KEBAKARAN TINGGI — MATIKAN SISTEM SEGERA!"

6. Kesesuaian Implementasi dengan Rancangan UML

Berdasarkan analisis menyeluruh terhadap source code sistem dan rancangan UML, berikut adalah hasil verifikasi kesesuaian:

Diagram UML	Status	Catatan
1. System Architecture	SESUAI	Seluruh komponen (ESP32, Firebase, Server, Dashboard, Telegram) dan koneksi antar komponen cocok. 5 modul backend sesuai.
2. Sequence Diagram	SESUAI	Seluruh 17 langkah dalam <code>processEarlyWarning()</code> diimplementasikan sesuai urutan yang digambarkan dalam diagram.
3. Class & Module Diagram	SESUAI	Semua class, property, method, dan relasi antar modul cocok. Terdapat 1 fungsi tambahan minor (<code>getDB()</code>) yang tidak ada di UML.
4. Flowchart Danger Classification	SESUAI	Logika klasifikasi identik. 2 perbedaan penamaan internal yang tidak berpengaruh pada fungsionalitas.

Kesimpulan

Implementasi source code sistem telah sesuai dengan rancangan UML Diagrams.

Perbedaan yang ditemukan hanya bersifat penamaan internal (`"none"` vs `"normal"` untuk default predictionRisk, dan `"high"` vs `"warning"` untuk humidityRisk) yang tidak mengubah perilaku sistem karena sudah di-mapping dengan benar sebelum penentuan level akhir.